Doc 3: Roadmap & Definition of Done

This roadmap dictates the execution order for the AI Agent. The AI must validate every requirement via tests and commit to Git before advancing.

Phase 1: Engine & Netcode Foundation

- [ ] Initialize `pnpm` workspace with `apps/server`, `apps/client`, and `packages/shared`.
- [ ] Set up Vitest/Jest for the workspace.
- [ ] Create a basic `HubRoom` on the Colyseus server.
- [ ] Implement the `terminal-kit` client to connect to the server and render a blank grid with an `@` player token.
- [ ] **AI Validation:** Write tests verifying Colyseus state updates when WASD inputs are simulated.
- [ ] **Git Control:** Commit to branch `feature/phase-1` and merge to `main`.

Phase 2: Map Parsing & Layout

- [ ] Create an LDtk `hub.json` map with `visuals` and `collisions` layers.
- [ ] Write an LDtk parser in `packages/map-engine` and validate it via unit tests.
- [ ] Server blocks player movement against the parsed collision array.
- [ ] Client renders the map using `terminal-kit` screen buffers and draws the UI sidebar for global chat.
- [ ] **AI Validation:** Write tests verifying movement logic throws/rejects when simulating a step into a wall tile.
- [ ] **Git Control:** Commit to branch `feature/phase-2` and merge to `main`.

Phase 3: Persistence & Player Accounts

- [ ] Initialize Prisma with SQLite.
- [ ] Create schemas for `User`, `Item`, and `Inventory`.
- [ ] Implement CLI login prompt in the client before establishing the WebSocket connection.
- [ ] Colyseus server fetches `walletBalance` from the database upon connection.
- [ ] **AI Validation:** Write integration tests mocking database calls to ensure user creation and wallet retrieval succeed.
- [ ] **Git Control:** Commit to branch `feature/phase-3` and merge to `main`.

Phase 4: Economy & Asynchronous Social

- [ ] Create `CasinoRoom` instance. Walking onto a specific map tile disconnects from Hub and connects to Casino.
- [ ] Implement server-side logic for betting/wallet deductions.
- [ ] Implement Prisma `ForumPost` model and the asynchronous Clubhouse BBS UI.
- [ ] **AI Validation:** Write unit tests verifying casino math (win/loss conditions) and database balance constraints (no negative balances).
- [ ] **Git Control:** Commit to branch `feature/phase-4` and merge to `main`.

Phase 5: Dungeons & PvE

- [ ] Integrate `rot-js` for procedural dungeon matrix generation in `packages/map-engine`.
- [ ] Create `DungeonRoom` instance that spins up a unique map on demand.
- [ ] Integrate `pathfinding` library for basic AI enemy movement toward the player.
- [ ] **AI Validation:** Write unit tests asserting that `rot-js` generates valid arrays and `pathfinding` returns valid coordinate paths without clipping walls.
- [ ] **Git Control:** Commit to branch `feature/phase-5` and merge to `main`.

Phase 6: Monetization & Polish

- [ ] Implement terminal image capability checks (Sixel/Kitty).
- [ ] Render `.png` digital billboards inline on the terminal canvas (with ASCII fallback).
- [ ] Finalize deployment configurations (Docker Compose, Environment variables).
- [ ] **Git Control:** Commit to branch `feature/phase-6`, tag release `v1.0.0`, merge to `main`.